

Lab 4 – Series de Televisión (con Apache Spark)

Big Data

27 de julio de 2026

Hoy vamos a analizar programas de TV para obtener (1) su rating promedio y (2) una lista de los capítulos con mejor rating. Usaremos datos de IMDb, que parecen así:

375095	9.3	The Wire	2002	tvSeries	null
4952	7.7	The Wire	2002	tvSeries	Ebb Tide (#2.1)
6208	8.2	The Wire	2002	tvSeries	The Detail (#1.2)
3997	8.8	The Wire	2002	tvSeries	Misgivings (#4.10)
3923	8.6	The Wire	2002	tvSeries	Home Rooms (#4.3)
...					

Las columnas son (1) el número de votos, (2) el voto promedio, (3) el nombre de la serie, (4) el año de la serie, (5) el tipo, (6) el nombre del capítulo; donde la columna (6) es nulo, la información de votación es el total para la serie (en IMDb, uno puede votar en la página de la serie o por un capítulo individual). Los datos están en el almacenamiento de HDFS del cluster: `hdfs://cm:9000/uhadoop/shared/imdb/imdb-ratings.tsv`. Lo que queremos producir es, por ejemplo:

```
The Wire#2002      -30- (#5.10)|Middle Ground (#3.11)      9.6      8.583
```

... donde las columnas son: (1) una llave para la serie (`nombre#año`), (2) una lista de los capítulos empatados con el mejor rating usando ‘|’ como un delimitador, (3) el mejor rating de un capítulo de la serie, (4) el rating promedio de todos los capítulos (excluyendo el valor de la serie; ej., la primera línea de la entrada arriba).¹

Ocuparemos Spark para el procesamiento. Hay varias opciones con las cuales ejecutar Spark, incluyendo Java 7, Java 8+, Python, Scala y R. Puedes entregar la tarea en el lenguaje que prefieras pero trabajaremos en el lab y en este enunciado con Java 8.²

- Descarga el proyecto `gdd-lab04.zip` de U-Cursos y ábrelo en Eclipse. En la carpeta `src` y el paquete `org.mdp.spark.cli` encontrarás algunas clases de utilidad y ejemplos para empezar. Revisa la clase `WordCountTask`. Más interesante aún es `AverageSeriesRating` que nos da la primera parte de la tarea que necesitamos: computar el rating promedio de la serie. La meta principal del lab será copiar y extender esta clase y computar la información sobre el mejor capítulo.
 - Estos ejemplos están en Java 8, pero en el paquete `org.mdp.spark.cli.j7` hay ejemplos para Java 7, en la carpeta `scala/` hay un ejemplo en Scala. También hay ejemplos para Python en la carpeta `py/` en la raíz del proyecto. Trabajaremos con los ejemplos de Java 8. La diferencia principal entre Java 8 y Java 7 es que se pueden usar expresiones de lambda para abreviar la definición de funciones.
- Antes de empezar con la extensión del código, ejecutaremos el ejemplo:
 - Primero, compilaremos el Jar:

¹Ojo que la salida real usará comas y paréntesis en lugar de tabs: no importa.

²Para aquellos que están familiarizados con Spark, hay que usar Spark Core y no Dataframes o Spark SQL, por ejemplo. El objetivo es aprender sobre Spark Core.

- En el proyecto en Eclipse, haz clic derecho en `build.xml`, luego **Run As ...**, y verifica que `dist` esté seleccionado y haz clic en **Run**. Si falla, puede ser que tengas que crear una carpeta nueva `dist` en la raíz del proyecto. Refresca el proyecto (F5) y asegurarte de tener el archivo JAR.
- Ahora tienes que copiar el JAR a `/data/2026/uhadoop/cc66i/USUARIO` en el servidor usando `scp`, `WinSCP`, etc. (como en los labs previos).
- Ejecuta (en un solo comando) desde la carpeta `/data/2026/uhadoop/cc66i/USUARIO`:
 - `spark-submit --master spark://cluster-01:7077 gdd-spark.jar AverageSeriesRating`
`hdfs://cm:9000/uhadoop/shared/imdb/imdb-ratings.tsv`
`hdfs://cm:9000/uhadoop2026/cc66i/USUARIO/series-avg/`
- Revisa los resultados en HDFS con `hdfs dfs -ls ...`, `hdfs dfs -cat ... | more`, etc.
- Ahora a la tarea principal: tu meta es usar Spark Core para computar una salida como en el ejemplo de la introducción. Tienes que copiar `AverageSeriesRating` a una nueva clase `InfoSeriesRating` y empezar a extenderla con la información del mejor capítulo. Al hacerlo, deberías seguir el ejemplo de `AverageSeriesRating` leyendo la entrada de HDFS y escribiendo la salida a HDFS. También deberías asegurarte de cachear cualquier RDD que uses más que una vez.
 - El JavaDoc para los RDDs puede resultar útil, en particular esta página que lista las varias transformaciones, acciones y métodos para cachear, y cuales argumentos toman:
<https://spark.apache.org/docs/latest/api/java/org/apache/spark/api/java/JavaRDD.html>
 - También se puede auxiliar de la guía de programación con RDD en Spark:
<https://spark.apache.org/docs/latest/rdd-programming-guide.html>
 - Hemos dejando un ejemplo aquí: `/uhadoop/shared/imdb/imdb-ratings-two.tsv`. Tiene dos series y una película irrelevante que debes filtrar. Puedes usarlo para probar tu código. En el caso de “The Wire”, debería dar un resultado parecido a él en la introducción del enunciado, pero la sintaxis puede ser un poco diferente, usando paréntesis y comas en lugar de tabs. No importa: no tienes que convertir la sintaxis a TSV.
- Una vez que la clase esté lista, deberías ejecutarla sobre los mismos datos (completos) que antes y revisar los resultados.
- ENTREGA (1) tu clase `InfoSeriesRating.java` [o tu alternativa en Java 7, Python o Scala], (2) la tupla de salida para la serie `American Crime Story#2016` y otra tupla de salida de tu elección (excluyendo `The Wire#2002` y `Arrested Development#2003`).
- OPCIONAL Puedes hacer la misma tarea en Hadoop y Pig, y comparar los tiempos.